

实验名称 实验3 复杂网络的度相关性

一、实验内容及结果

- 1、给定网络 1tux400.csv，计算节点的余平均度 $\langle k_{nn} \rangle_i$ 、度同配系数，余平均度 $\langle k_{nn} \rangle(k)$
- 2、画出 $\langle k_{nn} \rangle(k)$ -k 图，条件概率分布图（参考教材 P126 图 4-5）

源代码：

Task1:

```
import numpy as np
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
df= pd.read_csv("D:/Netdata/ex3/1tux400.csv")
G=nx.from_pandas_edgelist(df,'source','target','weight',create_using=nx.Graph())
print("节点度为: ",nx.degree(G,weight="weight"))
print("余平均度 $\langle k_{nn} \rangle_i$ 为:",nx.average_neighbor_degree(G,weight="weight"))
print("网络的度同配系数为:
",nx.degree_assortativity_coefficient(G,weight="weight"))
nx.draw(G,node_size=50,with_labels=True)
plt.show()
num_nodes = G.number_of_nodes()

def average_nearest_neighbor_degree(G):
    w_gd = dict(nx.degree(G, weight="weight"))

    k = set(degree for node, degree in w_gd.items())
    sorted_k = sorted(k)

    k_nn_k = []
    for ki in sorted_k:
        c = 0
        k_nn_i = 0
        for i in G.nodes():
            if w_gd[i] == ki:
```

```

        total_weighted_degree = sum([G[i][j]['weight'] * w_gd[j]
for j in list(nx.all_neighbors(G, i))])
        k_nn_i += total_weighted_degree / ki
        c += 1
        if c != 0:
            k_nn_k.append(k_nn_i / c)
        else:
            k_nn_k.append(0)

    return sorted_k, k_nn_k

r=nx.degree_assortativity_coefficient(G,weight="weight")
x,y=average_nearest_neighbor_degree(G)
for x, y in zip(x, y):
    print("度为 {} 的节点余平均度是 {}".format(x, y))

```

Task2:

```

x,y=average_nearest_neighbor_degree(G)
plt.figure(figsize=(12,4))
plt.plot(x,y,'ro')
plt.legend(loc=0)
plt.xlabel("$k$")
plt.ylabel("$k_{nn}()$")
plt.xscale("log")
plt.yscale("log")
plt.xlim([0,52])

def calculate_conditional_probability(G):
    conditional_prob = {}

    w_gd = dict(nx.degree(G, weight="weight"))
    k_values = set(w_gd.values())

    for k in k_values:
        p_k_prime = {}
        for node, degree in w_gd.items():
            if degree == k:
                neighbors = list(G.neighbors(node))
                neighbor_degrees = [w_gd[neighbor] for neighbor in neighbors]

                total_neighbor_weighted_degree = sum([G[node][neighbor]['weight'] * w_gd[neighbor] for neighbor in neighbors])
                for neighbor, neighbor_degree in zip(neighbors, neighbor_degrees):

```

```

        if neighbor_degree in p_k_prime:
            p_k_prime [neighbor_degree] += (G[node][neighbor]
['weight'] * w_gd[neighbor]) / total_neighbor_weighted_degree
        else:
            p_k_prime [neighbor_degree] = (G[node][neighbor][
'weight'] * w_gd[neighbor]) / total_neighbor_weighted_degree

    conditional_prob[k] = {k_prime: p_k_prime [k_prime] for k_prime i
n sorted(p_k_prime.keys())}

    return conditional_prob

```

```

conditional_prob = calculate_conditional_probability(G)
conditional_prob

```

```

from scipy.interpolate import interp1d
def plot_conditional_probability(conditional_prob):
    for k, p_k_prime in conditional_prob.items():
        degrees = list(p_k_prime.keys())
        probabilities = list(p_k_prime.values())
        if k in [6, 18, 29, 46]:
            # if k in [6]:
            degrees = list(p_k_prime.keys())
            probabilities = list(p_k_prime.values())
            f = interp1d(degrees, probabilities, kind='cubic')
            x_interp = np.linspace(min(degrees), max(degrees), 1000)
            y_interp = f(x_interp)
            plt.plot(x_interp, y_interp, label=f"k={k}", linewidth=2)
            # plt.plot(degrees, probabilities, label=f"k={k}")
            # for x, y in zip(degrees, probabilities):
            #     plt.text(x, y, f'({x}, {y:.2f})', fontsize=8, verticala
lignment='bottom')

    plt.xlabel(' k\''')
    plt.ylabel('P(k\'' | k)')
    plt.legend()
    plt.grid(True)
    plt.show()

```

```

plot_conditional_probability(conditional_probabilities)

```

运行结果（截图）：

Task1:

余平均度<kn>i:

```
余平均度<kn>i为: {'ALA1': 3.6666666666666665, 'ALA2': 3.3333333333333335, 'LEU301': 4.636363636363637, 'ALA3': 5.625, 'ASN253': 7.2, 'GLN299': 5.0
74074074074074, 'LEU300': 6.5, 'GLN4': 4.428571428571429, 'LEU9': 6.85, 'SER5': 7.142857142857143, 'GLN8': 6.7, 'VAL6': 8.990909090909092, 'ASP7':
7.1, 'ILE10': 7.578947368421052, 'ALA12': 5.75, 'ARG13': 8.15, 'VAL249': 7.391304347826087, 'TYR294': 7.518518518518518, 'LEU283': 8.6153846153846
15, 'PHE18': 7.055555555555555, 'ASP39': 7.083333333333333, 'ALA38': 7.2, 'PHE284': 7.377777777777778, 'PRO290': 5.833333333333333, 'LYS291': 6.29
4117647058823, 'ALA23': 6.8, 'ASN295': 6.166666666666667, 'THR246': 5.833333333333333, 'TYR248': 8.466666666666667, 'ASP250': 8.5, 'VAL252': 8.68
75, 'ILE297': 5.909090909090909, 'VAL269': 7.235294117647059, 'PRO281': 7.333333333333333, 'LEU282': 7.166666666666667, 'ASP11': 6.8333333333333
3, 'LYS15': 6.291666666666667, 'VAL16': 7.846153846153846, 'TYR17': 6.346153846153846, 'LEU256': 7.8, 'ILE264': 7.4, 'GLY19': 7.636363636363637,
'VAL20': 6.076923076923077, 'GLY41': 7.0, 'GLN42': 8.826808956521738, 'VAL266': 7.6, 'PHE40': 5.962962962962963, 'GLY14': 6.714285714285714, 'VAL2
51': 7.944444444444445, 'ALA254': 7.181818181818182, 'VAL258': 5.928571428571429, 'CYS261': 7.625, 'ILE262': 7.105263157894737, 'GLY263': 8.25, 'I
LE235': 7.578947368421052, 'CYS255': 8.466666666666667, 'ASP257': 7.571428571428571, 'LEU238': 8.235294117647058, 'THR265': 7.5, 'LEU222': 8.55555
555555555, 'PRO223': 7.5, 'ALA226': 7.2, 'ASP259': 5.8, 'SER260': 5.5, 'GLU232': 6.583333333333333, 'VAL233': 7.214285714285714, 'LEU79': 7.77777
777777778, 'ALA43': 7.157894736842105, 'ILE80': 8.166666666666666, 'ARG81': 7.36, 'ALA21': 7.571428571428571, 'THR21': 6.5625, 'PHE288': 8.481481
481481481, 'GLU237': 8.421052631578947, 'TRP267': 7.333333333333333, 'ALA270': 7.416666666666667, 'ASP271': 6.384615384615385, 'ASP285': 6.3571428
57142857, 'ARG26': 6.296296296296297, 'ASP23': 6.173913043478261, 'ASN32': 8.047619047619047, 'THR44': 7.5, 'TRP72': 7.53125, 'ASN287': 5.53333333
3333333, 'ASN289': 5.7894736842105265, 'GLN24': 7.217391304347826, 'ASN25': 9.181818181818182, 'LEU27': 8.791666666666666, 'GLY30': 7.0, 'LYS31':
5.684210526315789, 'GLU46': 7.625, 'PRO272': 6.857142857142857, 'ASP273': 6.428571428571429, 'PRO45': 5.933333333333334, 'ASN47': 5.61538461538461
5, 'SER48': 8.411764705882353, 'ALA33': 5.666666666666667, 'ALA73': 7.714285714285714, 'LYS78': 6.576923076923077, 'TYR68': 8.117647058823529, 'LE
U69': 9.071428571428571, 'ASN71': 7.6923076923076925, 'GLN75': 6.4375, 'ASN76': 7.611111111111111, 'THR28': 6.125, 'GLY65': 7.0, 'ALA66': 7.333333
333333333, 'MET49': 7.15, 'LYS50': 6.923076923076923, 'THR29': 5.2, 'ALA64': 5.333333333333333, 'ASP67': 7.230769230769231, 'VAL70': 7.8888888888888
889, 'TYR119': 6.870967741935484, 'GLY286': 6.285714285714286, 'TRP275': 5.25, 'GLY268': 7.727272727272727, 'SER274': 5.909090909090909, 'ARG27
6': 5.05, 'ALA277': 5.5, 'ILE35': 3.923076923076923, 'ALA34': 4.833333333333333, 'ILE36': 5.444444444444445, 'GLN37': 4.357142857142857, 'GLY77':
6.375, 'ILE123': 7.9523809523809526, 'ARG124': 7.666666666666667, 'ALA125': 9.1, 'GLY82': 9.444444444444445, 'ASP127': 9.80952380952381, 'THR236':
8.947368421052632, 'GLN74': 6.294117647058823, 'TRP126': 8.387096774193548, 'TRP51': 8.413793103448276, 'THR84': 7.533333333333333, 'ILE115': 8.26
6666666666667, 'MET116': 9.12, 'ARG118': 8.291666666666666, 'ILE120': 6.25, 'LYS122': 7.285714285714286, 'HIS83': 8.333333333333334, 'LEU85': 9.76
1904761904763, 'VAL128': 9.666666666666666, 'ALA167': 7.363636363636363, 'LYS168': 8.0, 'TYR170': 7.916666666666667, 'ASP52': 7.764705882352941,
'GLN58': 6.37037037037037, 'VAL86': 10.111111111111111, 'TRP94': 8.196078431372548, 'HIS111': 8.40625, 'ILE112': 8.619047619047619, 'PHE144': 7.826
086956521739, 'TRP87': 6.555555555555555, 'ASN130': 8.08695652173913, 'THR113': 7.0, 'THR114': 7.75, 'ALA53': 8.23076923076923, 'THR54': 6.5714285
71428571, 'GLN90': 5.761904761904762, 'PRO92': 6.809523809523809, 'SER93': 6.357142857142857, 'GLU55': 5.5, 'PRO56': 7.090909090909091, 'SER57':
6.777777777777778, 'GLY59': 6.333333333333333, 'ASN60': 4.866666666666666, 'PHE63': 6.166666666666667, 'HIS88': 6.896551724137931, 'TYR152': 7.947
368421052632, 'ALA156': 9.75, 'VAL95': 6.3076923076923075, 'VAL107': 7.722222222222222, 'MET108': 10.344827586206897, 'ASN101': 6.631578947368421,
'LYS109': 8.090909090909092, 'THR159': 8.133333333333333, 'ALA160': 8.368421052631579, 'LEU104': 6.875, 'THR142': 8.826808956521738, 'VAL143': 8.3
33333333333334, 'ASN145': 7.8125, 'ASN146': 6.111111111111111, 'VAL147': 7.642857142857143, 'ILE148': 7.3076923076923075, 'SER89': 7.6, 'LEU91':
5.5, 'VAL96': 5.833333333333333, 'ASN62': 4.5, 'PHE61': 4.666666666666667, 'THR117': 6.2, 'ASN166': 5.846153846153846, 'GLY121': 4.75, 'ASN172':
8.095238095238095, 'SER206': 8.066666666666666, 'GLN207': 7.272727272727275, 'THR208': 9.052631578947368, 'ALA234': 9.272727272727273, 'VAL129':
9.888888888888889, 'GLU131': 7.125, 'ASP173': 8.0, 'TYR174': 7.3076923076923075, 'ASN175': 7.75, 'LEU176': 7.470588235294118, 'ALA163': 8.666666666
666666, 'ASP164': 5.105263157894737, 'LEU169': 8.238095238095237, 'PHE157': 7.787878787878788, 'ALA132': 8.352941176470589, 'ARG140': 7.9, 'ILE15
3': 10.555555555555555, 'GLU150': 6.7272727272727275, 'ASP151': 7.555555555555555, 'PRO154': 7.235294117647059, 'ILE155': 7.461538461538462, 'ILE1
71': 8.8, 'PHE133': 7.72, 'LEU139': 6.55, 'ASN134': 5.705882352941177, 'GLN141': 5.636363636363637, 'ILE98': 4.642857142857143, 'SER97': 4.875, 'T
HR103': 6.538461538461538, 'ASN106': 6.5, 'THR99': 5.5, 'ASP100': 6.083333333333333, 'LYS101': 6.214285714285714, 'THR105': 7.357142857142857, 'AS
N102': 6.111111111111111, 'GLY149': 5.142857142857143, 'ARG158': 9.222222222222221, 'ALA162': 7.285714285714286, 'ARG161': 7.166666666666667, 'PRO
165': 7.25, 'ASP202': 7.333333333333333, 'GLY203': 9.5, 'ILE204': 7.4375, 'THR230': 5.071428571428571, 'PRO231': 7.125, 'GLY205': 8.71428571428571
4, 'TRP194': 7.310344827586207, 'VAL199': 6.523809523809524, 'ILE201': 8.380952380952381, 'VAL191': 8.105263157894736, 'LYS183': 6.692307692307692
5, 'ILE187': 8.055555555555555, 'ALA186': 6.923076923076923, 'ARG190': 7.523809523809524, 'SER138': 6.5, 'GLU135': 6.0, 'ASP136': 4.625, 'GLY137':
5.333333333333333, 'PRO182': 7.444444444444445, 'THR184': 8.0, 'SER185': 6.888888888888889, 'LYS189': 7.25, 'VAL188': 6.705882352941177, 'LYS193':
6.230769230769231, 'GLY198': 5.666666666666667, 'PRO200': 6.818181818181818, 'ARG195': 6.538461538461538, 'ALA228': 7.545454545454546, 'LYS192':
6.727272727272727, 'HIS209': 6.863636363636363, 'ASP177': 7.933333333333334, 'SER178': 6.666666666666667, 'ILE218': 8.272727272727273, 'LEU225':
7.882352941176471, 'LEU210': 7.631578947368421, 'SER211': 5.272727272727275, 'ASP239': 9.166666666666666, 'ILE240': 7.08, 'ALA241': 7.22222222222
222, 'LYS181': 5.294117647058823, 'ALA216': 7.0, 'SER217': 8.076923076923077, 'ALA179': 6.875, 'SER180': 6.5, 'ALA221': 7.6, 'ASP219': 7.57142857
1428571, 'ASN224': 7.769230769230769, 'SER227': 6.416666666666667, 'GLY215': 5.166666666666667, 'ALA220': 8.142857142857142, 'ALA196': 5.625, 'GLY
229': 6.714285714285714, 'ALA197': 5.2, 'ALA212': 6.125, 'GLY213': 4.666666666666667, 'GLN214': 4.5, 'ALA243': 4.777777777777778, 'THR279': 5.5,
'THR280': 6.5625, 'GLY242': 5.75, 'ASP247': 5.8125, 'THR244': 5.181818181818182, 'SER278': 5.166666666666667, 'SER245': 5.75, 'ALA296': 5.53846153
8461538, 'VAL298': 7.357142857142857, 'PRO292': 6.1}
```

共 301 个节点，每个节点所有邻居节点的平均度为余平均度<kn>i；对于此数据集，所有度的计算都要考虑权重。

度同配系数：

网络的度同配系数为：**0.20366393269906685**

网络度同配系数为 0.2，代表节点倾向连接具有相似度的节点，但同配趋势不十分强烈。

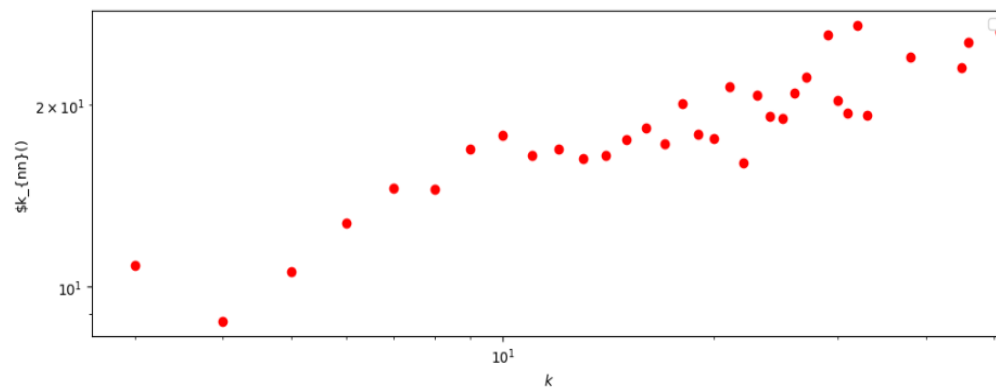
余平均度 $\langle k_{nn} \rangle(k)$:

度为 3 的节点余平均度是	10.833333333333334
度为 4 的节点余平均度是	8.75
度为 5 的节点余平均度是	10.6
度为 6 的节点余平均度是	12.715686274509803
度为 7 的节点余平均度是	14.55952380952381
度为 8 的节点余平均度是	14.494047619047619
度为 9 的节点余平均度是	16.92063492063492
度为 10 的节点余平均度是	17.808333333333334
度为 11 的节点余平均度是	16.52631578947369
度为 12 的节点余平均度是	16.903508771929825
度为 13 的节点余平均度是	16.265734265734267
度为 14 的节点余平均度是	16.517006802721088
度为 15 的节点余平均度是	17.488888888888887
度为 16 的节点余平均度是	18.302083333333332
度为 17 的节点余平均度是	17.22689075630252
度为 18 的节点余平均度是	20.07264957264957
度为 19 的节点余平均度是	17.845864661654133
度为 20 的节点余平均度是	17.596153846153847
度为 21 的节点余平均度是	21.3968253968254
度为 22 的节点余平均度是	16.06060606060606
度为 23 的节点余平均度是	20.724637681159418
度为 24 的节点余平均度是	19.183333333333334
度为 25 的节点余平均度是	18.98
度为 26 的节点余平均度是	20.884615384615383
度为 27 的节点余平均度是	22.208333333333332
度为 29 的节点余平均度是	26.11206896551724
度为 30 的节点余平均度是	20.366666666666667
度为 31 的节点余平均度是	19.38709677419355
度为 32 的节点余平均度是	27.078125
度为 33 的节点余平均度是	19.242424242424242
度为 38 的节点余平均度是	24.026315789473685
度为 45 的节点余平均度是	23.044444444444444
度为 46 的节点余平均度是	25.434782608695652
度为 51 的节点余平均度是	26.470588235294116

将网络中所有节点可能的度按序排列, 共 34 种, 余平均度 $\langle k_{nn} \rangle(k)$ 是对度相同的节点的 $\langle k_{nn} \rangle_i$ 再次取平均。

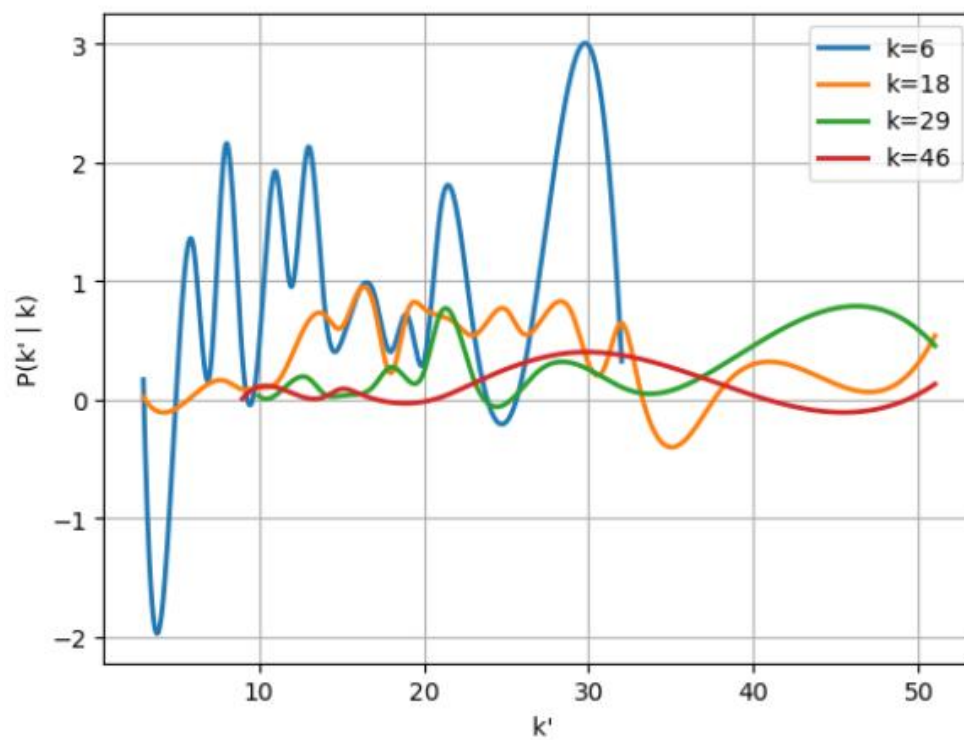
Task2:

$\langle k_{nn} \rangle(k) - k$:



$\langle k_{nn} \rangle(k)$ 图像如上图所示，可以看出递增趋势，表明网络具有一定程度的同配性质。

$K=[6, 18, 29, 46]$ 时的 $P(k' | K)$:



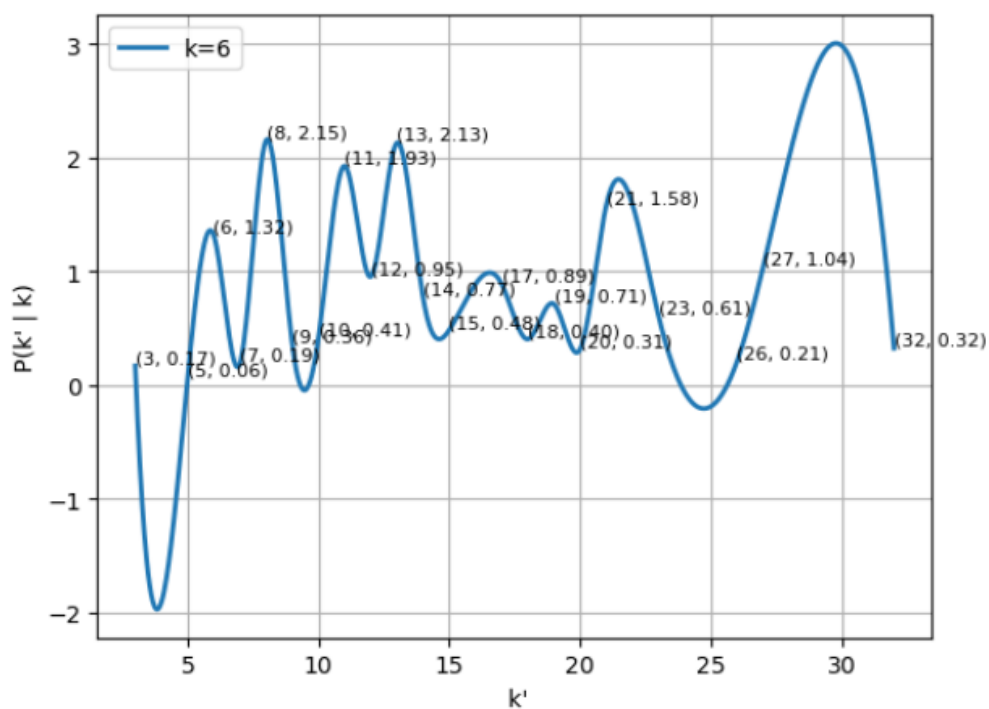
从 34 个可能取值的 K 中选取其中 4 个绘制图像，任意改变 K 值，可以获得不同 K 下的条件概率。

K=6 时的条件概率及图像：

```

6: {3: 0.16827852998065762,
5: 0.056179775280898875,
6: 1.3172595520421606,
7: 0.18894927536231884,
8: 2.1518119791074035,
9: 0.363854938890002,
10: 0.411509900990099,
11: 1.9263021954369348,
12: 0.9530814035225774,
13: 2.1302426140087216,
14: 0.7704661182922052,
15: 0.4838709677419355,
17: 0.8919464318473674,
18: 0.40372670807453415,
19: 0.7125,
20: 0.3125,
21: 1.5789161034200478,
23: 0.6097602161431949,
26: 0.20634920634920634,
27: 1.0392617602774104,
32: 0.32323232323232326},
:

```



K=6 时，所有度为 6 节点邻居节点可能的度和出现概率如上，图像通过插值方法连接离散点绘制形成。